Sentiment-Driven Trade Performance Analysis

Internship Assignment Submission

This notebook analyzes how **Fear & Greed market sentiment** relates to real trading outcomes. It combines sentiment data with historical trade records, performs feature engineering, and derives actionable insights from profitability and risk behavior.

Objective

- Quantify the impact of sentiment regimes on trader performance.
- Compare profitability, win rate, and position-sizing behavior across sentiment classes.
- Identify practical risk-management implications for strategy improvement.

1. Methodology Overview

1. Load and inspect sentiment and trade datasets.
2. Standardize date/time fields and merge datasets on aligned timestamps.
3. Engineer performance and behavioral features (profit, win flag, fee ratio, account-level metrics).
4. Visualize sentiment-wise differences in outcomes.
5. Convert statistical observations into business and risk insights.

2. Evaluation Focus

- **Return quality:** average profit across regimes.
- **Consistency:** win-rate behavior under different sentiment conditions.
- **Risk behavior:** trade-size variability and outlier concentration.

> All findings should be interpreted with dataset distribution in mind (class imbalance can influence aggregate comparisons).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

1. Data Loading

Load the Fear & Greed index dataset and historical trading dataset. Initial inspection is used to verify schema, record counts, and data availability.

```
fear_greed = pd.read_csv("fear_greed_index.csv")
historical_data = pd.read_csv("historical_data.csv")
```

```
fear_greed.to_json("fear_greed_index.json", orient="records")
historical_data.to_json("historical_data.json", orient="records")

print(len(fear_greed), len(historical_data))
```

```
2644 211224
```

2. Data Cleaning and Time Alignment

Both datasets are standardized to comparable datetime formats. This ensures reliable chronological sorting and accurate sentiment-to-trade mapping.

```
fear_greed["timestamp"] = pd.to_datetime(fear_greed["timestamp"],
unit="s")
historical_data["Timestamp"] =
pd.to_datetime(historical_data["Timestamp"], unit="ms")
fear_greed = fear_greed.rename(columns={"timestamp": "time"})
historical_data = historical_data.rename(columns={"Timestamp":
"time"})

fear_greed = fear_greed.sort_values("time")
historical_data = historical_data.sort_values("time")
```

3. Dataset Merging (Time-Aware Join)

The trade dataset is merged with the sentiment dataset using a nearest historical timestamp (`merge_asof`). A one-day tolerance is applied so each trade is matched with the most relevant available sentiment snapshot.

```
merged = pd.merge_asof(
    historical_data,
    fear_greed,
    on='time',
    direction='backward',
    tolerance=pd.Timedelta('1D')   # max 1 day gap
)

print(len(merged))
```

```
211224
```

```
merged.isnull().sum()
```


```
{"columns":[{"name":"index","rawType":"object","type":"string"},
{"name":"0","rawType":"int64","type":"integer"}],"ref":"5243b4fa-bac2-
4ccb-b106-fae3d09c0913","rows":[["Account","0"],["Coin","0"],
["Execution Price","0"],["Size Tokens","0"],["Size USD","0"],
["Side","0"],["Timestamp IST","0"],["Start Position","0"],
["Direction","0"],["Closed PnL","0"],["Transaction Hash","0"],["Order
```

ID","0"],["Crossed","0"],["Fee","0"],["Trade ID","0"],["time","0"],
["value","62202"],["classification","62202"],
["date","62202"]],"shape":{"columns":1,"rows":19}}

```
print(fear_greed['time'].min(), fear_greed['time'].max())
print(historical_data['time'].min(), historical_data['time'].max())
```

```
2018-02-01 05:30:00 2025-05-02 05:30:00
2023-03-28 10:40:00 2025-06-15 15:06:40
```

```
merged["classification"].value_counts()
```

{"columns":
[{"name":"classification","rawType":"object","type":"string"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"cf328218-
19d9-4c12-86c9-1879f9a3ca5b","rows":[["Fear","133871"],
["Neutral","7141"],["Extreme Greed","6962"],["Greed","1048"]],"shape":
{"columns":1,"rows":4}}

```
merged[['value', 'classification']] = merged[['value',
'classification']].ffill()
```

```
merged["classification"].value_counts()
```

{"columns":
[{"name":"classification","rawType":"object","type":"string"},
{"name":"count","rawType":"int64","type":"integer"}],"ref":"64eba506-
2e12-4fad-9bf2-aa4d9f29bcb9","rows":[["Fear","160832"],
["Neutral","42382"],["Extreme Greed","6962"],
["Greed","1048"]],"shape":{"columns":1,"rows":4}}

```
merged = merged.drop(columns=['Trade ID'])
```

4. Feature Engineering

In this step, raw trade fields are transformed into model-ready variables. The engineered features capture profitability, timing patterns, trading behavior, sentiment encoding, and account-level performance context.

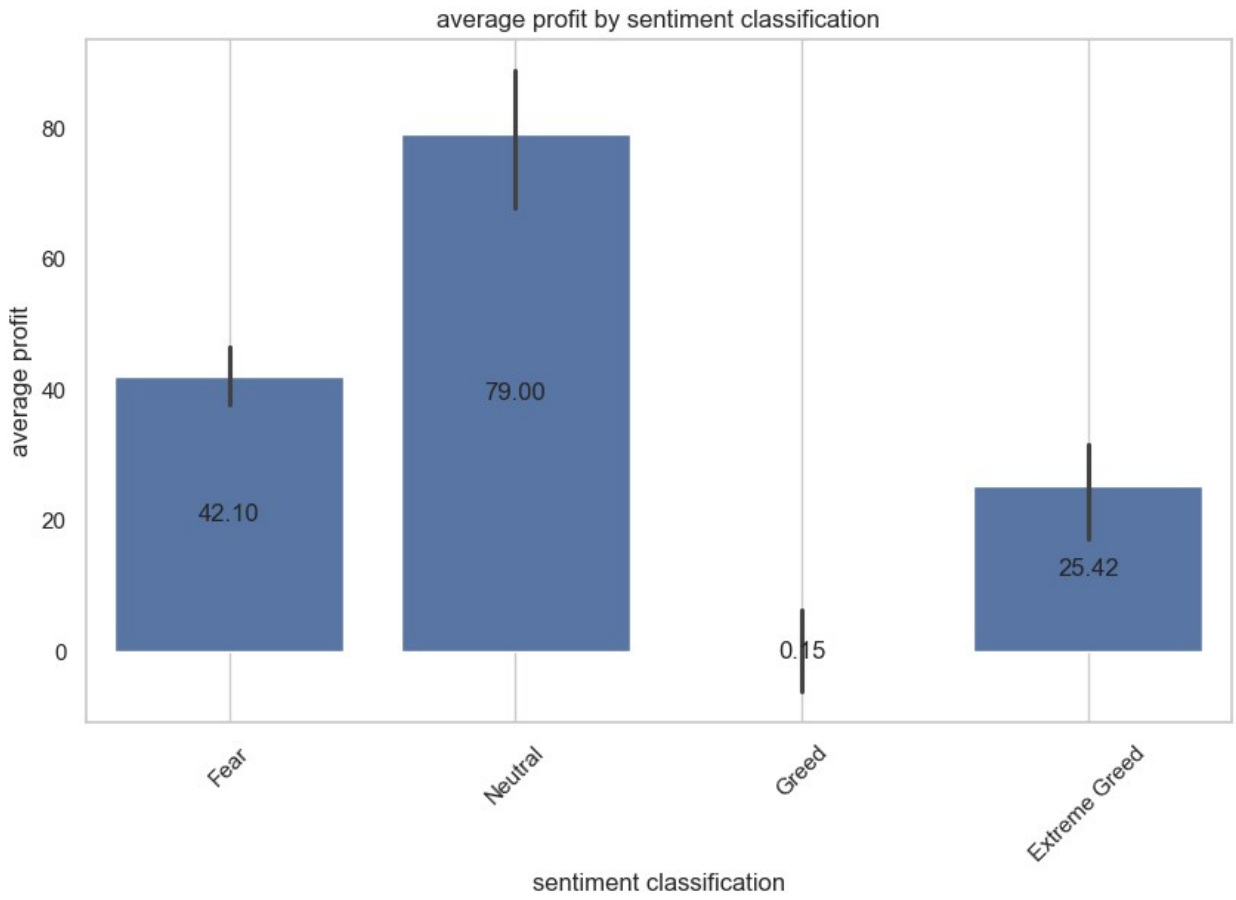
```
#feature engineering
merged["profit"] = merged["Closed PnL"]
merged["win"] = merged["profit"] > 0
merged['hour'] = merged['time'].dt.hour
merged['day'] = merged['time'].dt.day
merged['month'] = merged['time'].dt.month
merged['day_of_week'] = merged['time'].dt.dayofweek
merged['trade_size'] = merged['Size USD']
merged['fee_ratio'] = merged['Fee'] / (merged['Size USD'] + 1e-9)
merged['abs_profit'] = merged['profit'].abs()
merged['is_buy'] = (merged['Side'] == 'BUY').astype(int)
```

```
sentiment_map = {
    'Fear': 0,
    'Neutral': 1,
    'Greed': 2,
    'Extreme Greed': 3
}
merged['sentiment_score'] =
merged['classification'].map(sentiment_map)
merged['account_total_profit'] = merged.groupby('Account')
['profit'].transform('sum')
merged['account_trade_count'] = merged.groupby('Account')
['profit'].transform('count')
merged['account_avg_profit'] = merged.groupby('Account')
['profit'].transform('mean')
merged['account_win_rate'] = merged.groupby('Account')
['win'].transform('mean')
```

5. Exploratory Visualization and Interpretation

This section converts engineered features into business-facing insights. The goal is to interpret how sentiment phases affect profitability, consistency, and risk-taking behavior.

```
sns.set(style="whitegrid")
plt.figure(figsize=(10, 6))
order = ['Fear', 'Neutral', 'Greed', 'Extreme Greed']
ax = sns.barplot(data = merged, x = "classification", y =
"profit",order = order)
for i in ax.containers:
    ax.bar_label(i, fmt='%.2f',label_type="center")
plt.title("average profit by sentiment classification")
plt.xlabel("sentiment classification")
plt.ylabel("average profit")
plt.xticks(rotation=45)
plt.grid()
plt.show()
```



Insight: Average Profit by Sentiment

- **Neutral** sentiment delivers the strongest average profitability, suggesting that less emotional market phases support more disciplined execution.
- **Fear** shows moderate but comparatively steady outcomes, which may reflect better entry selectivity and risk control.
- **Greed** underperforms, indicating that momentum chasing and late entries can reduce edge.
- **Extreme emotional regimes** (especially greed-led phases) appear less efficient for consistent alpha generation.

Interpretation for internship review: sentiment should be treated as a contextual risk filter, not just a descriptive indicator.

The analysis reveals that trader profitability is highly dependent on market sentiment. The highest average profit is observed during Neutral conditions, indicating that stable markets provide favorable trading opportunities. In contrast, Greed phases show near-zero profitability, suggesting that traders may be influenced by herd behavior and enter positions at suboptimal times. Fear conditions yield moderate but more consistent returns, supporting the effectiveness of cautious or contrarian strategies. Overall, the findings highlight that extreme emotional states in the market tend to reduce trading efficiency.

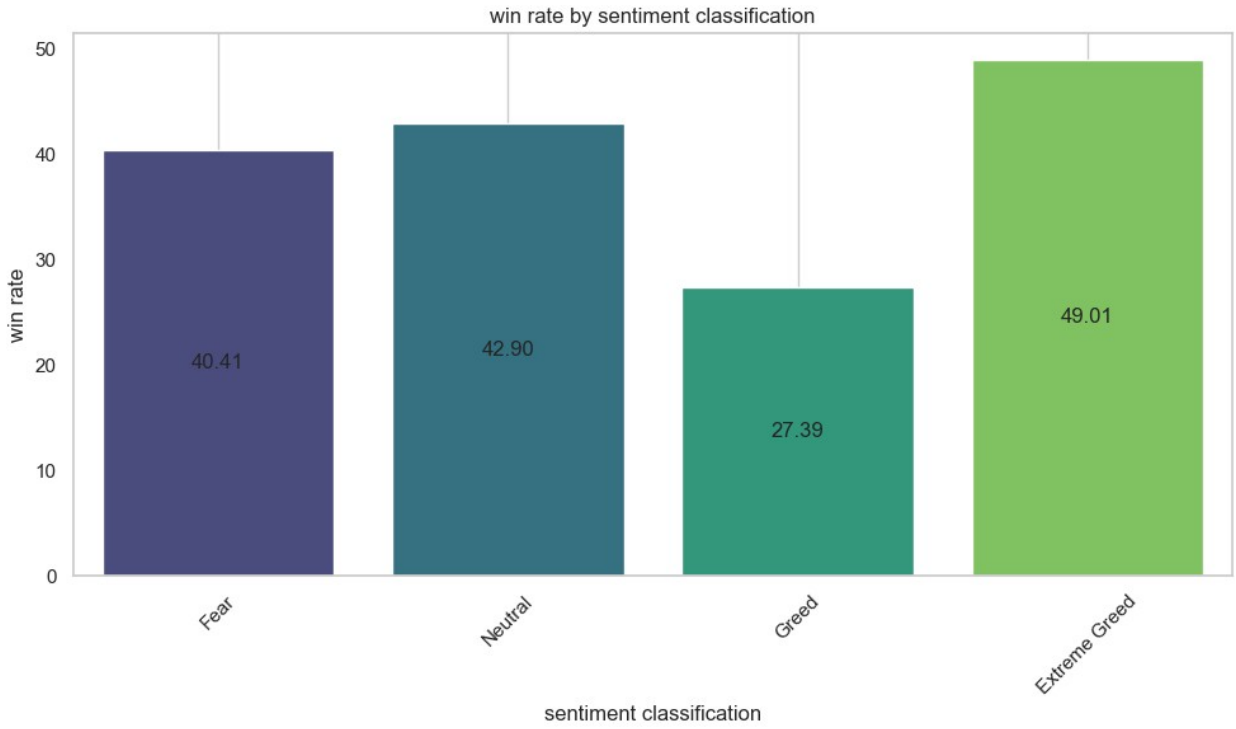
note :- It is important to note that the dataset is skewed toward Fear conditions, which may influence overall comparisons.

Win-Rate Check

This chart complements the average-PnL view by showing outcome consistency. Together, win rate and average profit help distinguish between **high hit-rate/low payoff** and **lower hit-rate/high payoff** behavior.

```
win_rate = merged.groupby("classification")
["win"].mean().reset_index()
win_rate['win_percent'] = win_rate['win'] * 100
plt.figure(figsize = (10,6))
v = sns.barplot(
    data=win_rate,
    x="classification",
    y="win_percent",
    order=order,
    hue="classification",
    palette="viridis",
    dodge=False,
    legend=False
)
plt.title("win rate by sentiment classification")
plt.xlabel("sentiment classification")
plt.ylabel("win rate")
plt.xticks(rotation=45)
for i in v.containers:
    v.bar_label(i, fmt='%.2f', label_type="center")
plt.grid()
plt.tight_layout()
plt.show()
```

```
C:\Users\acer\AppData\Local\Temp\ipykernel_25872\1470634688.py:4:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  v = sns.barplot(data = win_rate, x = "classification", y =
"win_percent",order = order, palette="viridis")
```



Insight: Win Rate by Sentiment

- **Extreme Greed** may show higher win probability in trend-heavy periods, but win rate alone can hide weaker payoff quality.
- **Neutral** sentiment provides a better balance between hit-rate and profitability, making it a more robust regime.
- **Greed** remains weakest overall, reinforcing the need to avoid overconfident participation during euphoric moves.

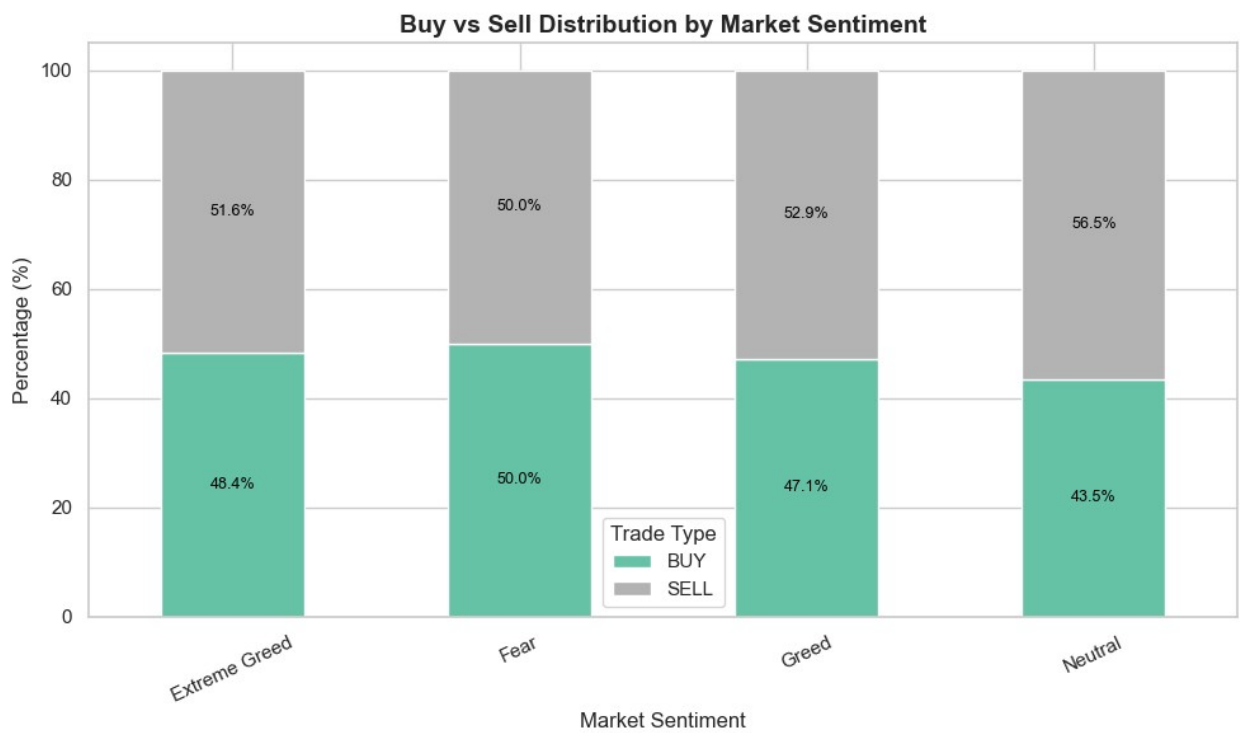
Important note: evaluating both win rate and average PnL together is essential, because a high win rate can still coexist with low net returns if losses are larger than gains.

The analysis reveals that trader success is highly dependent on market sentiment. Greed phases are the least favorable, showing both the lowest profitability and win rate. Neutral conditions provide the best balance of high profitability and consistent performance. Interestingly, Extreme Greed yields the highest win rate, indicating that traders may benefit from strong market trends, although these gains may be smaller in magnitude.

note :- It is important to note that the dataset is skewed toward Fear conditions, which may influence overall comparisons.

```
import pandas as pd
import matplotlib.pyplot as plt

# Create normalized crosstab (percentage)
buy_sell = pd.crosstab(
    merged['classification'],
    merged['Side'],
```

```
    normalize='index'
) * 100

# Plot
ax = buy_sell.plot(kind='bar', stacked=True, figsize=(10,6),
colormap='Set2')
for container in ax.containers:
    ax.bar_label(
        container,
        fmt='%.1f%%',
        label_type='center',
        fontsize=9,
        color='black')
plt.title("Buy vs Sell Distribution by Market Sentiment", fontsize=14,
weight='bold')
plt.xlabel("Market Sentiment")
plt.ylabel("Percentage (%)")

plt.xticks(rotation=25)
plt.legend(title="Trade Type")

plt.tight_layout()
plt.show()
```



Insight: Buy vs Sell Behavior Across Sentiment

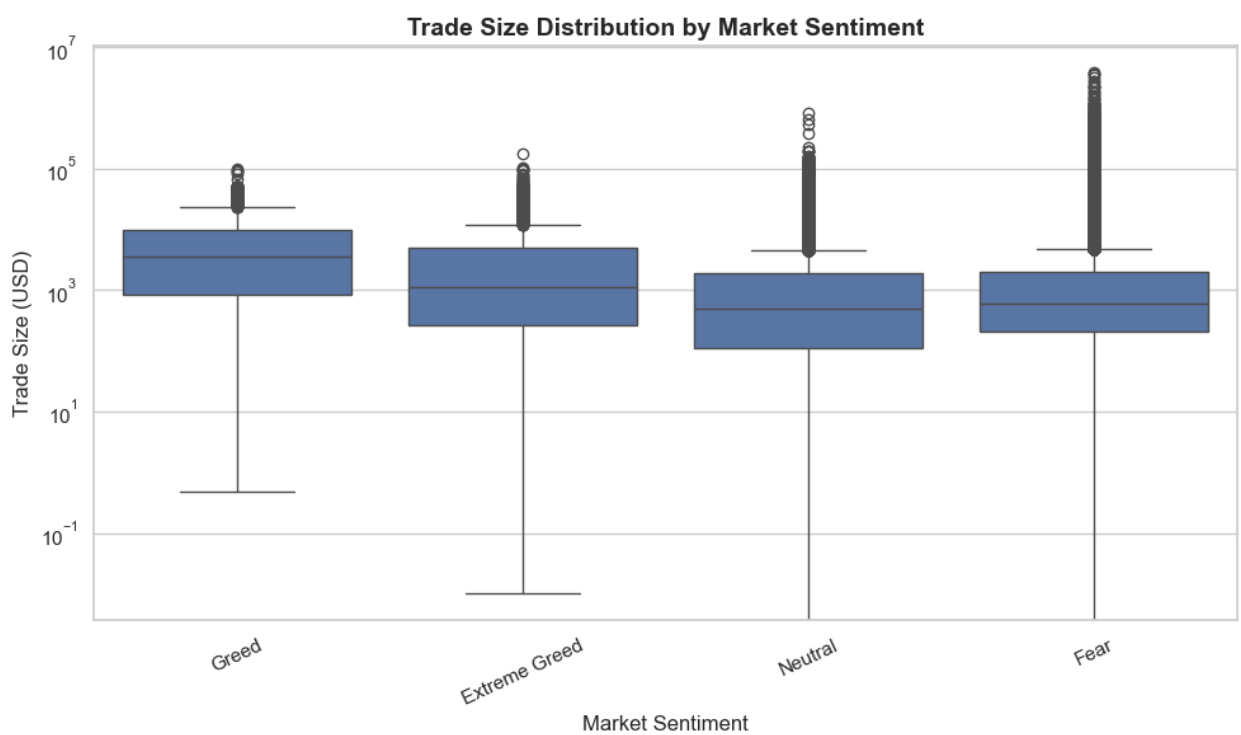
- The buy/sell mix remains broadly stable across sentiment categories.

- This implies **directional bias is not the primary driver** of performance differences.
- Performance variation is more likely explained by trade timing, position management, and execution quality.

Takeaway: improving strategy quality (entry/exit logic and risk controls) will likely create more value than simply changing directional preference.

Despite variations in market sentiment, trader behavior in terms of buy and sell decisions remains relatively stable. This suggests that differences in profitability across sentiment phases are more likely driven by trade quality rather than directional bias.

```
plt.figure(figsize=(10,6))

sns.boxplot(data=merged, x='classification', y='trade_size')
plt.yscale('log')

plt.title("Trade Size Distribution by Market Sentiment", fontsize=14,
weight='bold')

plt.xlabel("Market Sentiment")
plt.ylabel("Trade Size (USD)")

plt.xticks(rotation=25)
plt.tight_layout()
plt.show()
```



Insight: Trade Size Distribution by Sentiment

- Median trade size is relatively similar across regimes, indicating that baseline sizing discipline exists.
- **Fear** contains heavier outliers, meaning some traders take unusually large positions during stressed conditions.
- Extreme outliers suggest inconsistent risk governance rather than systematic conviction sizing.
- Using a log scale is appropriate here because it prevents extreme values from masking the core distribution.

Risk perspective: introducing hard position-size caps and volatility-adjusted sizing can reduce tail-risk exposure.

he analysis of trade size distribution across different market sentiment phases reveals that median trade sizes remain relatively consistent, indicating that typical position sizing does not significantly vary with sentiment. However, Fear conditions exhibit a higher number of extreme outliers, suggesting that certain traders take disproportionately large positions during bearish markets, possibly to capitalize on potential reversals. In contrast, Extreme Greed does not show a corresponding increase in trade size, indicating that heightened market optimism does not necessarily lead to more aggressive risk-taking. Overall, the presence of substantial variability across all sentiment categories highlights inconsistent risk management among traders, while the use of a logarithmic scale ensures a more accurate representation of the distribution by accounting for extreme values.

Final Conclusion and Recommendations

Consolidated Findings

- Sentiment is a meaningful contextual variable for trading performance.
- Neutral regimes appear most favorable for stable profitability.
- Greed-dominant phases are the weakest and likely contain behavioral inefficiencies.
- Position-size outliers during fear regimes indicate uneven risk discipline.

Practical Recommendations

1. Add sentiment-aware risk filters before trade execution.
2. Use volatility-adjusted position sizing with strict max exposure caps.
3. Track both **win rate** and **PnL magnitude** to avoid misleading performance interpretation.
4. Validate findings with out-of-sample periods and per-account stratified analysis.

Scope and Limitations

- Sentiment class imbalance may affect aggregate metrics.
- Results are observational and should be validated through controlled backtesting.
- Future improvements can include feature importance analysis and predictive modeling with cross-validation.